08 图

- 最短路径
- 拓扑排序
- 关键路径

08 图

- 最短路径
- 拓扑排序
- 关键路径

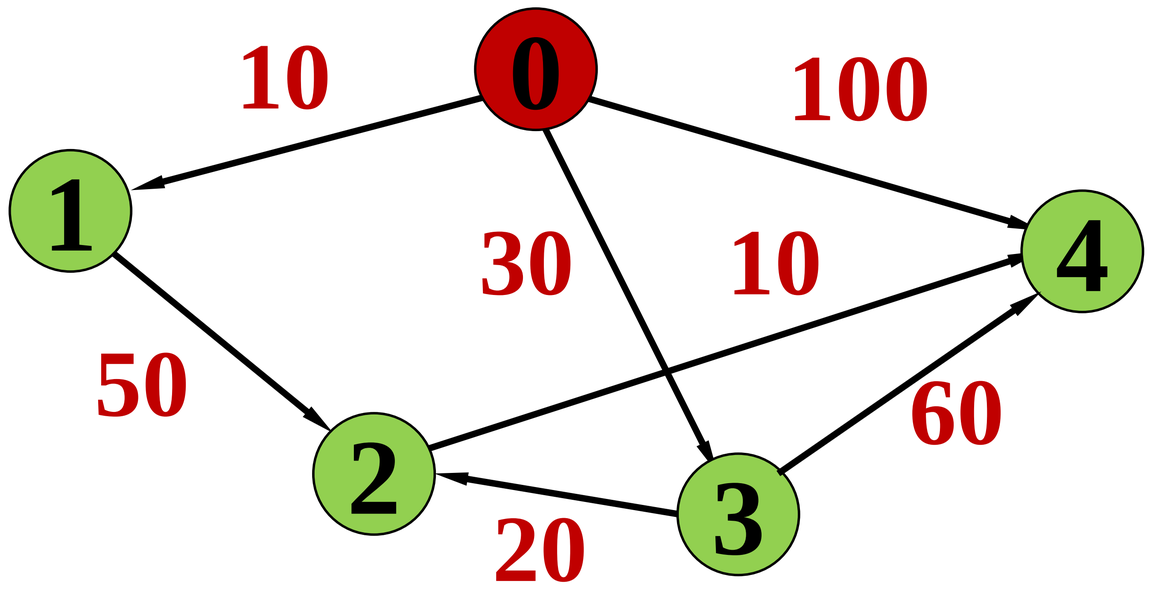
最短路径问题

- **最短路径(shortest path)问题：从带权图中某一顶点(称为源点)到达另一顶点(称为终点)的路径不止一条，找到其中各边上的权值总和最小的路径**
  - **非负权值的单源最短路径问题：Dijkstra算法**
  - **任意权值的单源最短路径问题：Bellman-Ford算法**
  - **所有顶点之间的最短路径问题：Floyd算法**

非负权值的单源最短路径问题

- 问题定义： 给定一个带权有向图D与源点v， 求从v到D中其它顶点的最短路径，限定各边上的权值大于或等于0
- 为求得这些最短路径， Dijkstra提出按路径长度的递增次序， 逐步产生最短路径的算法
  - 首先求出长度最短的一条最短路径
  - 再参照它求出长度次短的一条最短路径
  - 依此类推，直到从顶点v到其它各顶点的最短路径全部求出为止

Dijkstra算法过程示例



源点	终点	最短路径	路径长度
v_0	v_1	(v_0,v_1)	10
v_0	v_2	—— -> (v_0,v_1,v_2) -> (v_0,v_3,v_2)	∞ -> 60 -> 50
v_0	v_3	(v_0,v_3)	30
v_0	v_4	(v_0,v_4) -> (v_0,v_3,v_4) -> (v_0,v_3,v_2,v_4)	100 -> 90 -> 60

Dijkstra算法基本思路

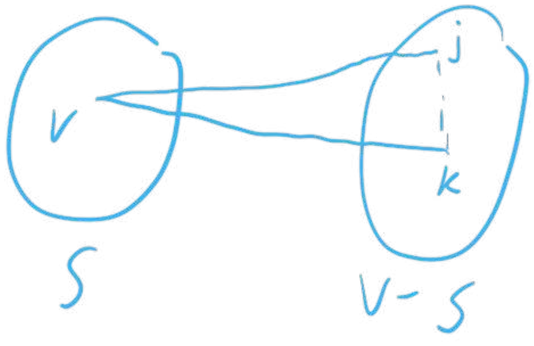
- 把图中顶点集合V分成两组，令S表示已求出最短路径的顶点集合，其余的尚未确定最短路径的顶点集合为V-S
- 按最短路径长度的递增次序逐个地把V-S中的顶点加入S中，直至从v出发可以到达的所有顶点都在S中，在上述过程中，需要始终保持
  - 从v到S中各顶点的最短路径长度都不经过V-S中的任何顶点
  - 每个顶点维持一个距离dist，S中的顶点的dist就是从v到此顶点的最短路径长度，而V-S中的顶点的dist是从v到此顶点的只包括S中的顶点为中间顶点的当前最短路径长度

Dijkstra算法基本思路

- 初始时，S = {v}，即S中只包含源点v，dist(v)为零；V-S包括其它所有顶点，对其中的每个顶点w，若图中有边<v, w>，则dist(w)为<v, w>的权值，否则，dist(w)为∞
- 步骤1：从V-S中取出一个顶点k加入S，满足dist(k)最小
- 步骤2：若用k做中间顶点，使得从v到顶点j (j ∈ V-S)的dist(j)比原来不经过顶点k的dist(j)更短，则要修改dist(j)
- 迭代执行步骤1和2，直到图中所有顶点都包含在S中，或再也没有可加入S的顶点存在

Dijkstra算法正确性分析

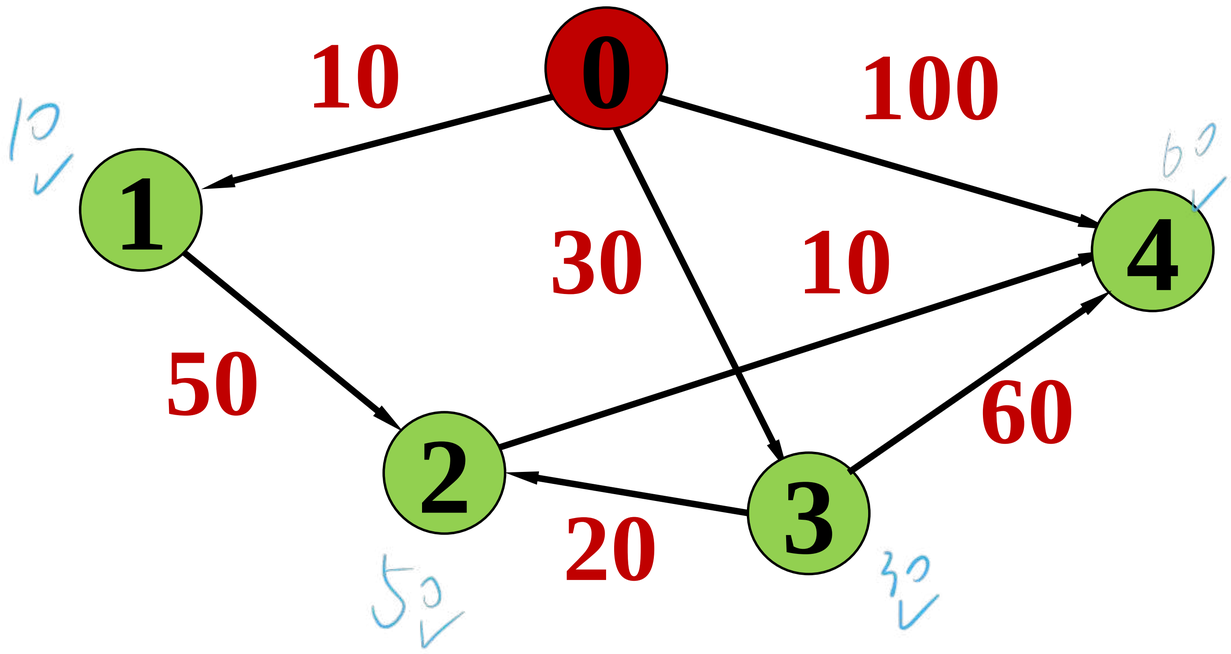
- 证明“对于从V-S中取出的dist(k)最小的顶点k，dist(k)就是从v到k的最短路径的长度”
- 若dist(k)不是从v到k的最短路径的长度，则有一条从v经过V-S某些顶点到达k的路径，其长度比dist(k)更短
- 设经过的V-S中的第一个顶点是j，则v到j的距离小于v到k的距离，与k是V-S中dist最小的顶点相矛盾



Dijkstra算法实现

```
void ShortestPath(Graph <string, float> &G, int v, float Dist[], int Pre[], int S[]) {
    int i, j, k, n = G.VerticesNumber( );  float Min, w;
    for (i = 0; i < n; i++) {  // Dist、Pre和S数组初始化
        Dist[i] = G.GetWeight(v, i);  S[i] = 0;
        if (Dist[i] < MAXINT)  Pre[i] = v;  else  Pre[i] = -1;
    }
    S[v] = 1;  Pre[v] = -1;  // 顶点v加入集合S中
    for (i = 0; i < n - 1; i++) {  // 从顶点v确定n-1条路径
        Min = MAXINT;  k = -1;
        for (j = 0; j < n; j++)  // 选择不在集合S中具有最短路径的顶点k
            if (S[j] == 0 && Dist[j] < Min) { Min = Dist[j];  k = j; }
        if (k == -1) break;  // 找不到新的最短路径
        S[k] = 1;  // 将顶点u加入S集合，表示它已在最短路径上
        for (j = G.GetFirstNeighbor(k); j != -1; j = G.GetNextNeighbor(k, j)) {  // 修改
            w = G.GetWeight(k, j);
            if (S[j] == 0 && w < MAXINT && Dist[k] + w < Dist[j]) {
                Dist[j] = Dist[k] + w;  Pre[j] = k;
            }
        }
    }
}
```

Dijkstra算法中各辅助数组的最终结果



读取源点0到终点的最短路径的方法：以顶点4为例

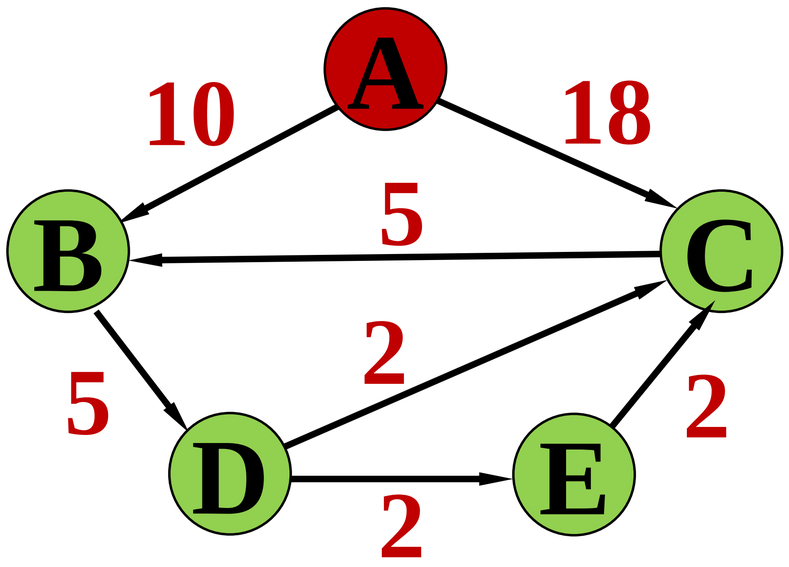
pre[4]=2, pre[2]=3, pre[3]=0,

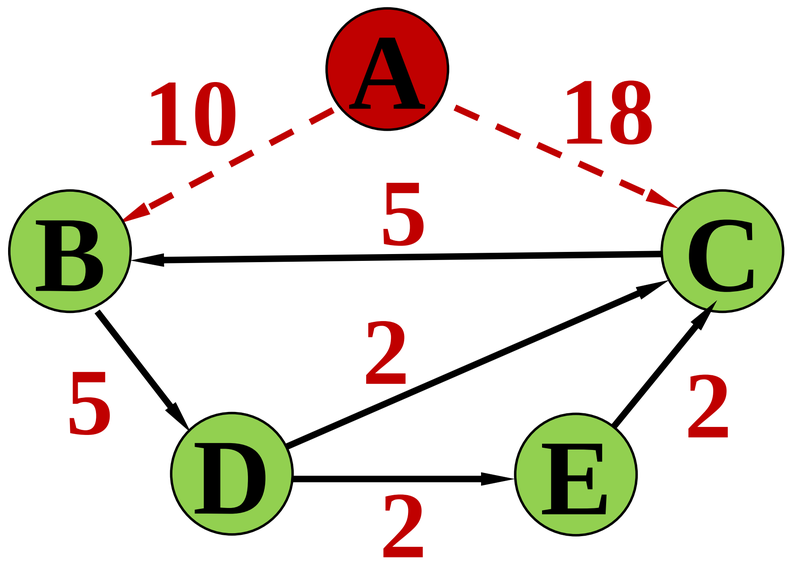
逆序即得到路径0, 3, 2, 4，即源点0到终点4的最短路径

序号	顶点0	顶点1	顶点2	顶点3	顶点4
S	1	1	1	1	1
Dist	0	10	50	30	60
Pre	-1	0	3	0	2

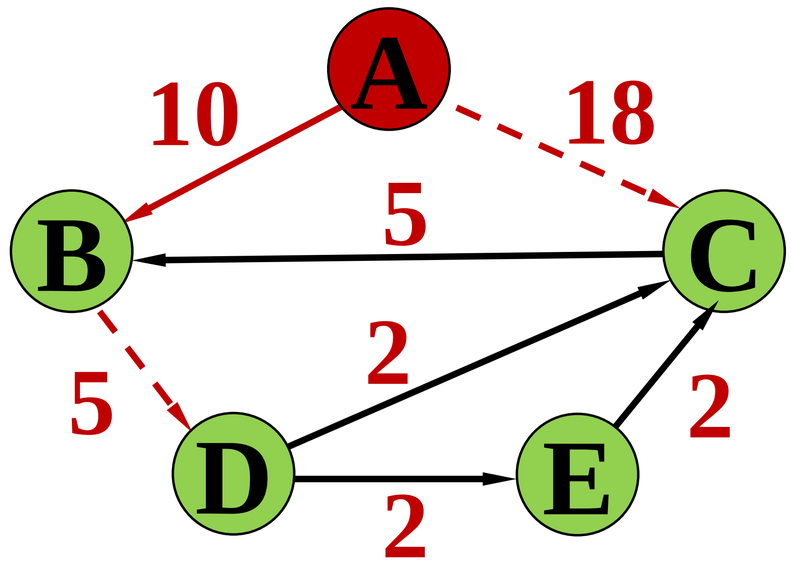
练习1

- 按Dijkstra算法计算从A到各个顶点的最短路径

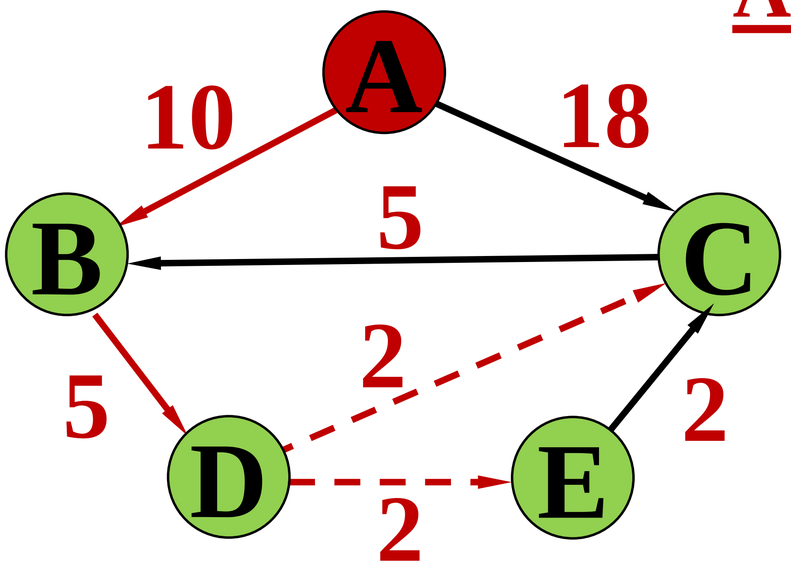




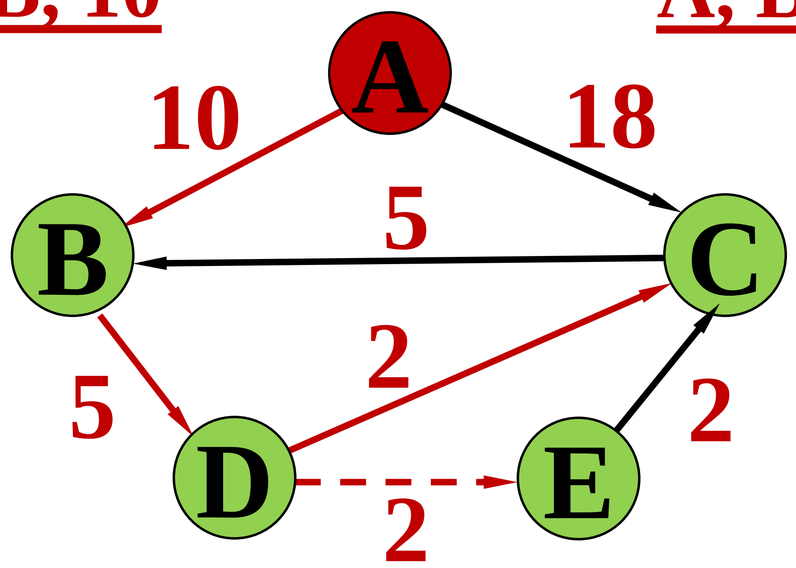
A, B, 10



A, B, D, 15



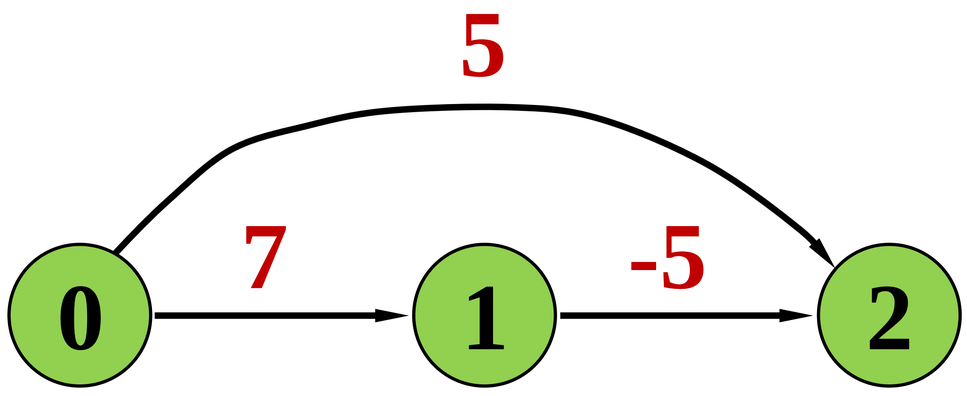
A, B, D, C, 17



A, B, D, E, 17

任意权值的单源最短路径问题

- 带权有向图G的某几条边或所有边的长度可能为负值，利用Dijkstra算法，不一定能得到正确的结果

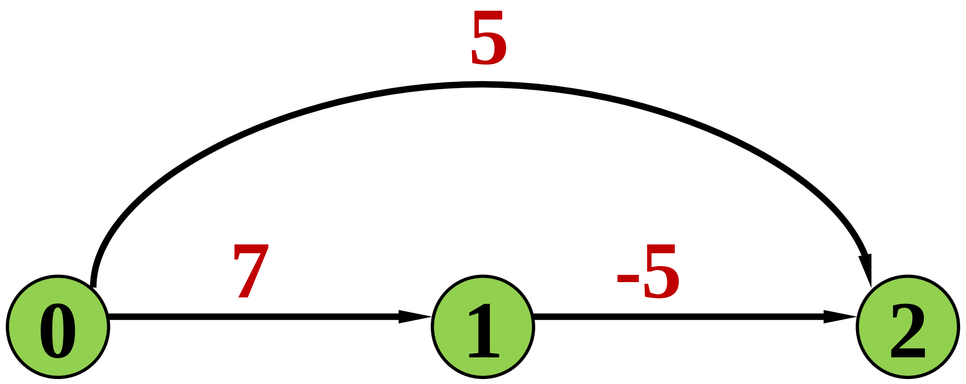


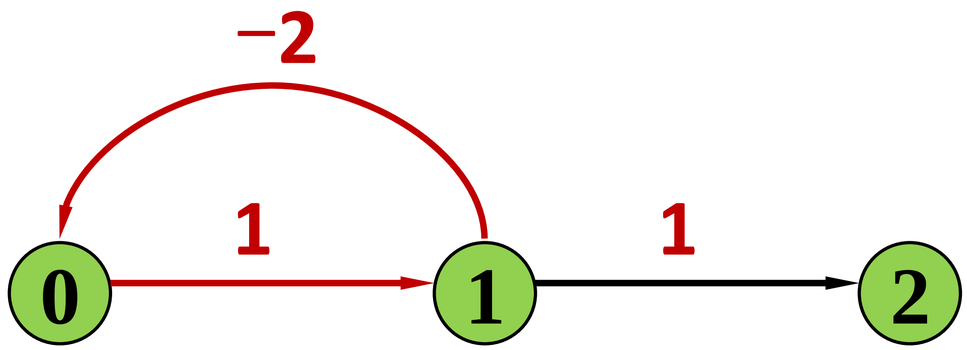
若设源点v=0，则使用Dijkstra算法所得结果为

选取顶点	顶点0			顶点1			顶点2		
	S[0]	Dist[0]	Pre[0]	S[1]	Dist[1]	Pre[1]	S[2]	Dist[2]	Pre[2]
0	1	0	-1	0	7	0	0	5	0
2	1	0	-1	0	7	0	1	5	0
1	1	0	-1	1	7	0	1	5	0

Bellman-Ford算法

- **Bellman和Ford提出了从源点逐次经过其它顶点，以缩短到达终点的最短路径长度的算法**
- **该算法有一个限制条件，即要求图中不能包含权值总和为负的回路**





一直自环

无下界

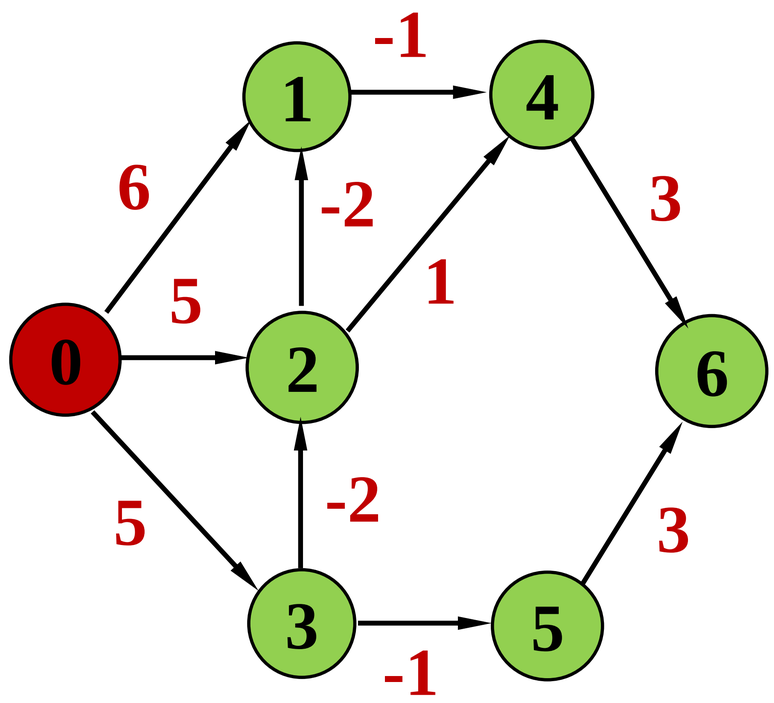
Bellman-Ford算法基本思路

- 当有n个顶点的图中没有权值总和为负的回路时，图中任意两个顶点之间若存在最短路径，此路径最多有n-1条边
- 以此为依据，Bellman-Ford算法构造一个从顶点v到终点u的最短路径长度的序列$dist^1[u], dist^2[u], \ldots, dist^{n-1}[u]$
  - $dist^1[u]$是从v到u的只经过一条边的最短路径长度，即Edges[v][u]
  - $dist^2[u]$是从v出发最多经过两条边到达u的最短路径的长度
  - $dist^3[u]$是从v出发最多经过三条边到达u的最短路径的长度
  - …
  - $dist^{n-1}[u]$是从v出发最多经过n-1条边到达u的最短路径的长度
  - 算法的最终目的是计算出$dist^{n-1}[u]$

Bellman-Ford算法基本思路

- 可以用递推方式计算$dist^k[u]$
  - $dist^1[u] = Edges[v][u]$
  - $dist^k[u] = min\{dist^{k-1}[u], min\{dist^{k-1}[j] + Edges[j][u]\}\}$
- 设已经求出$dist^{k-1}[j]$, $j = 0, 1, \ldots, n-1$，即从源点v最多经过k-1条边到达终点j的最短路径的长度
  - 计算$min\{dist^{k-1}[j] + Edges[j][u]\}$可得从源点v经过各顶点j，最多经过k条边到达终点u的最短路径的长度，用它与$dist^{k-1}[u]$比较，取小者作为$dist^k[u]$的值

Bellman-Ford算法过程示例



k	$dist^k[0]$	$dist^k[1]$	$dist^k[2]$	$dist^k[3]$	$dist^k[4]$	$dist^k[5]$	$dist^k[6]$
1	0	6	5	5	∞	∞	∞
2	0	3	3	5	5	4	∞
3	0	1	3	5	2	4	7
4	0	1	3	5	0	4	5
5	0	1	3	5	0	4	3
6	0	1	3	5	0	4	3

Bellman-Ford算法实现

```
void BellmanFord(Graph <string, float> &G, int v, float dist[ ], int pre[ ]) {
    int n = G.VerticesNumber( );
    for (int i = 0; i < n; i++) {
        dist[i] = G.GetWeight(v, i);
        if (i != v && dist[i] < MaxValue)  pre[i] = v;  else  pre[i] = -1;
    }
    for (int k = 2; k < n; k++)  // n-2次迭代
        for (int u = 0; u < n; u++)  // 更新dist[u]
            if (u != v)
                for (i = 0; i < n; i++) {  // 其他顶点存在到u的边
                    float w = G.GetWeight(i, u);
                        if (w < MaxValue && dist[i] + w < dist[u]) {
                            dist[u] = dist[i] + w;  pre[u] = i;
                        }
                }
}
```

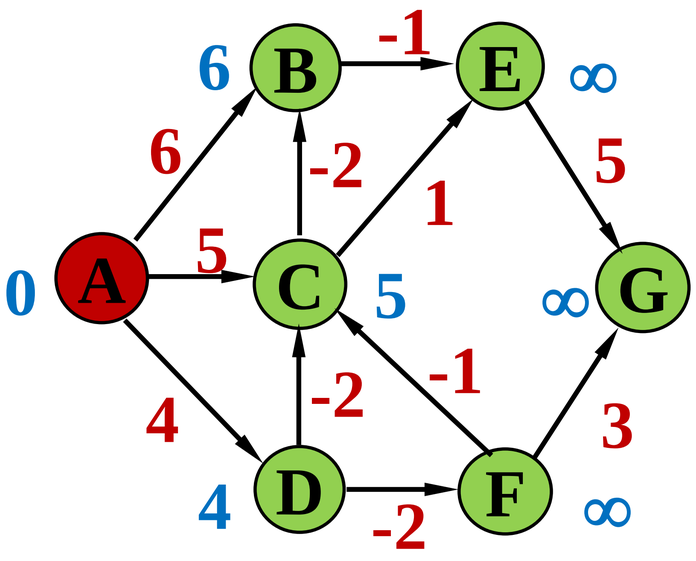
找入度

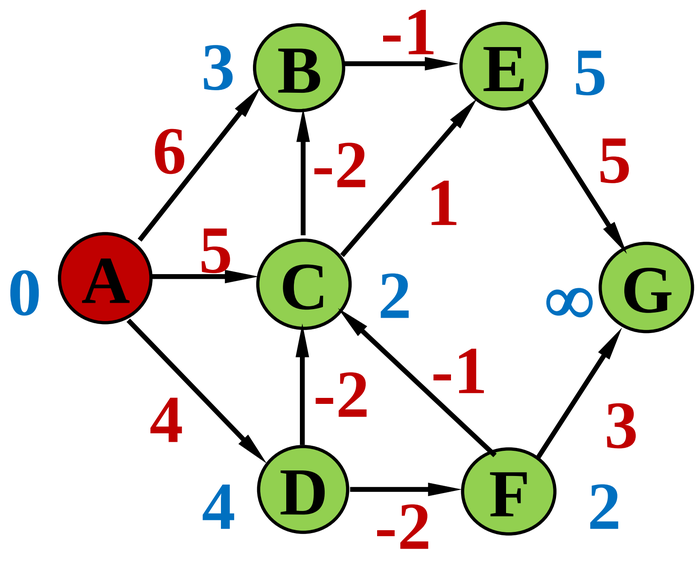
Bellman-Ford算法性能分析

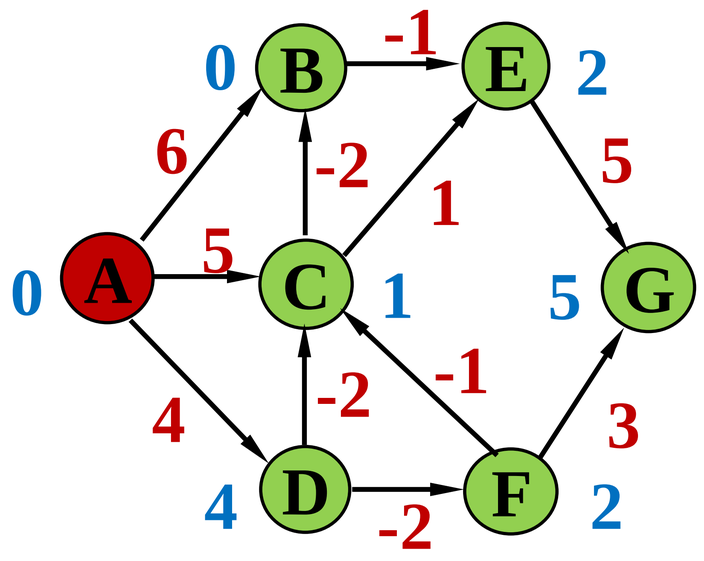
- 用邻接矩阵作为存储结构，总时间为$O(|V|^3)$ 三层循环
- 用邻接表作为存储结构，可通过优化把内层的两个for循环变成一个循环，每次访问|E|条边，总时间为O(|V|*|E|)
  - 邻接表要略做修改，把各个边链表连接成一个链表，原来内层的两个for循环变成一个遍历链表的循环

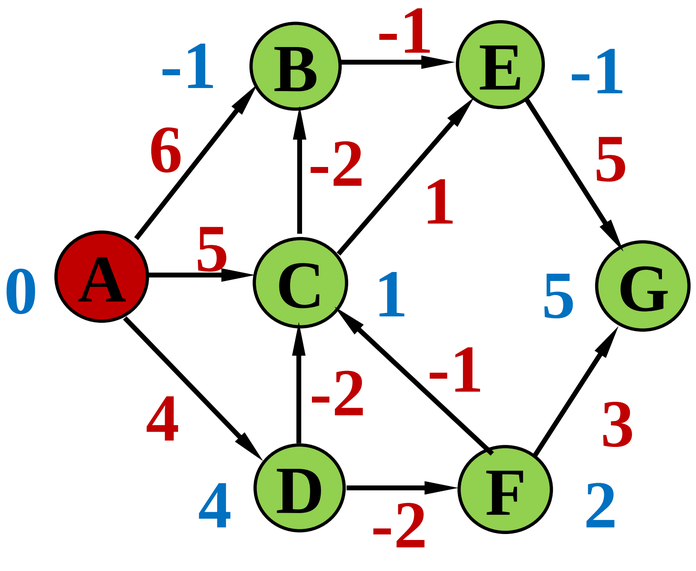
练习2

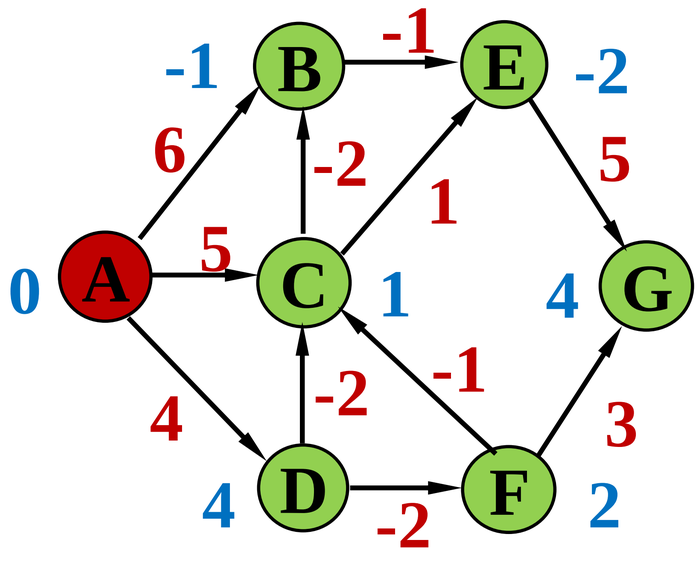
- 按Bellman-Ford算法计算从A到各个顶点的最短路径

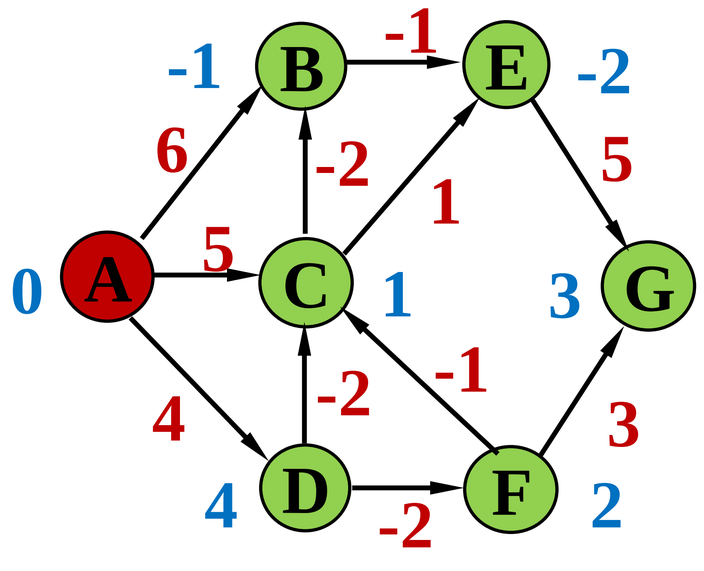












所有顶点之间的最短路径问题

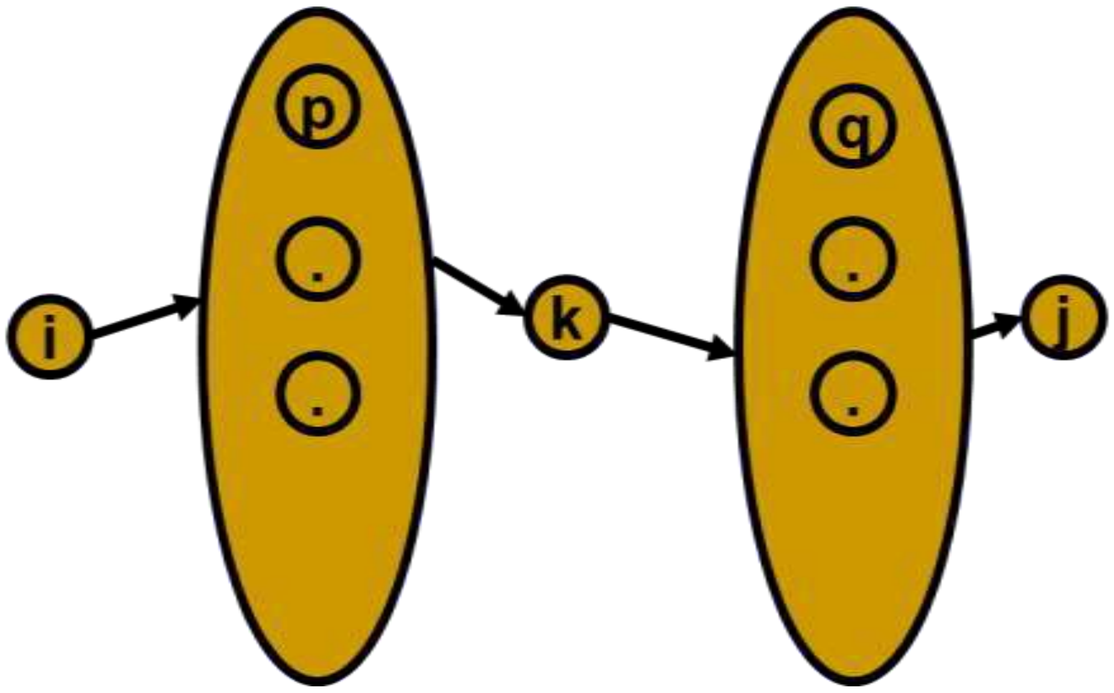
- 问题定义：已知一个各边权值均大于0的带权有向图，对每一对顶点$v_i \neq v_j$，求出v_i与v_j之间的最短路径及其长度
- 一种方法是轮流以每个顶点为源点，重复执行Dijkstra算法|V|次，总时间是$O(|V|^3)$
- 另一种更直接的方法是Floyd算法
  - 定义一个n阶方阵序列：$A^{(-1)}, A^{(0)}, A^{(1)}, \ldots, A^{(n-1)}$
  - $A^{(-1)}[i][j] = Edges[i][j]$
  - $A^{(k)}[i][j] = \min\{A^{(k-1)}[i][j], A^{(k-1)}[i][k] + A^{(k-1)}[k][j]\}, k = 0, 1, \ldots, n-1$
  - $A^{(0)}[i][j]$是从v_i到v_j，中间可能经过v_0的最短路径的长度
  - $A^{(k)}[i][j]$是从v_i到v_j，中间顶点的序号不大于k的最短路径的长度
  - $A^{(n-1)}[i][j]$是从v_i到v_j的最短路径的长度

Floyd算法基本思路

- 递推地产生一个矩阵序列$A^{(-1)}, A^{(0)}, \ldots, A^{(k)}, \ldots, A^{(n-1)}$，其中$A^{(-1)}$为给定的权值邻接矩阵，$A^{(k)}(i, j)$ $(0 \le i, j \le n-1)$表示从顶点i到顶点j的中间顶点序号不大于k的最短路径长度
- 若从i到j的最短路径没有中间顶点，则对于$0 \le k \le n-1$，有 $A^{(k)}(i, j) = A^{(-1)}(i, j) = Edges[i][j]$
- 递推地产生$A^{(0)}, A^{(1)}, \ldots, A^{(k)}, \ldots, A^{(n-1)}$的过程就是逐步允许越来越多的顶点作为路径的中间顶点，直至找到所有允许的中间顶点，求出最短路径

Floyd算法基本思路

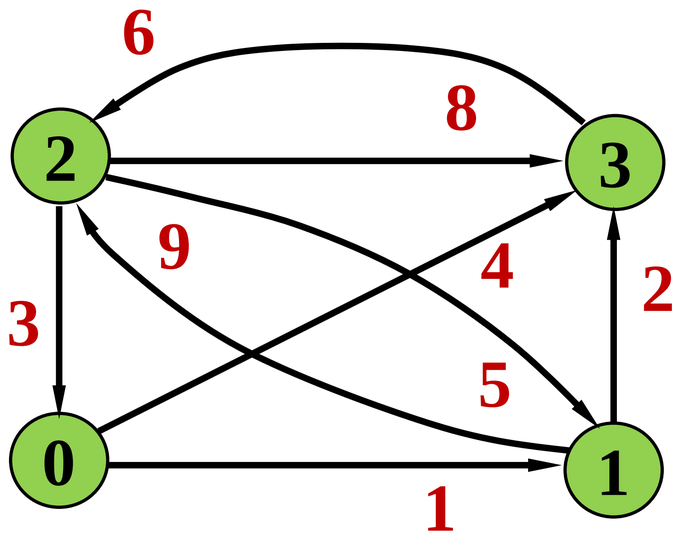
- 假设已求出$A^{(k-1)}(i, j)$ $(0 \le i, j \le n-1)$，如何求出$A^{(k)}(i, j)$？
  - 如果从顶点i到j的最短路径不经过顶点k，则从i到j的中间顶点序号不大于k的最短路径长度就是$A^{(k-1)}(i, j)$，即$A^{(k)}(i, j) = A^{(k-1)}(i, j)$
  - 如果从顶点i到顶点j的最短路径经过顶点k，则这条路径是由i到k和由k到j的两条路径所组成，由于$A^{(k-1)}(i, k) + A^{(k-1)}(k, j) < A^{(k-1)}(i, j)$，则$A^{(k)}(i, j) = A^{(k-1)}(i, k) + A^{(k-1)}(k, j)$



Floyd算法实现

```
void Floyd(Graph <string, float> &G, float a[ ][ ], int path[ ][ ]) {
    // a[i][j]是顶点i和j之间的最短路径长度
    // path[i][j]是相应路径上顶点j的前一顶点的顶点号   从i到j的路径上
    int i, j, k, n = G.VerticesNumber( );
    for (i = 0; i < n; i++)  // 矩阵a与path初始化
        for (j = 0; j < n; j++) {
            a[i][j] = G.GetWeight(i, j);
            if (i != j && a[i][j] < MaxValue) path[i][j] = i; else path[i][j] = 0;
        }
    for (k = 0; k < n; k++)  // 针对每一个k，产生a(k)及path(k)
        for (i = 0; i < n; i++)
            for (j = 0; j < n; j++)
                if (a[i][k] + a[k][j] < a[i][j]) {  // 缩短路径长度，经过k到j
                    a[i][j] = a[i][k] + a[k][j]; path[i][j] = path[k][j];
                }
}
```

Floyd算法过程示例



以 Path(3)为例，对最短路径读法加以说明
从A(3)知，顶点1到0的最短路径长度为a[1][0] = 11，其最短路径看path[1][0] = 2，path[1][2] = 3，path[1][3] = 1，表示顶点0 ← 顶点2 ← 顶点3 ← 顶点1，即从顶点1到顶点0最短路径为<1, 3, 2, 0>

	$A^{(-1)}$				$A^{(0)}$				$A^{(1)}$				$A^{(2)}$				$A^{(3)}$			
	0	*1*	*2*	*3*	*0*	*1*	*2*	*3*	*0*	*1*	*2*	*3*	*0*	*1*	*2*	*3*	*0*	*1*	*2*	*3*
0	0	1	∞	4	0	1	∞	4	0	1	10	3	0	1	10	3	0	1	9	3
1	∞	0	9	2	∞	0	9	2	∞	0	9	2	12	0	9	2	11	0	8	2
2	3	5	0	8	3	4	0	7	3	4	0	6	3	4	0	6	3	4	0	6
3	∞	∞	6	0	∞	∞	6	0	∞	∞	6	0	9	10	6	0	9	10	6	0

	$Path^{(-1)}$				$Path^{(0)}$				$Path^{(1)}$				$Path^{(2)}$				$Path^{(3)}$			
	0	*1*	*2*	*3*	*0*	*1*	*2*	*3*	*0*	*1*	*2*	*3*	*0*	*1*	*2*	*3*	*0*	*1*	*2*	*3*
0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	1	1	0	0	3	1
1	0	0	1	1	0	0	1	1	0	0	1	1	2	0	1	1	2	0	3	1
2	2	2	0	2	2	0	0	0	2	0	0	1	2	0	0	1	2	0	0	1
3	0	0	3	0	0	0	3	0	0	0	3	0	2	0	3	0	2	0	3	0

Floyd算法性能分析

- 三层循环，时间为$O(|V|^3)$
- Floyd算法允许图中有带负权值的边，但不允许有权值总和为负的回路
- Floyd算法也可用于带权无向图，即把带权无向图可以看作是有往返两条边的有向图

练习3

- 某乡有A, B, C, D四个村庄，若村庄i和村庄j有路可通，则w_{ij}为从村庄i到村庄j的距离。如何选定一个村庄建立中心俱乐部，使得离中心俱乐部最远的村庄离俱乐部尽可能近？
  - 应用Floyed算法得到每对村庄间的最短路径
  - 找出从一个村庄出发到其他各村庄的最短路径中最长路径
  - 在这4条最长路径中找出最短的一条，其出发点即为所求

计算传递闭包

- **设A为有向图G的邻接矩阵**
  - **G的传递闭包矩阵**

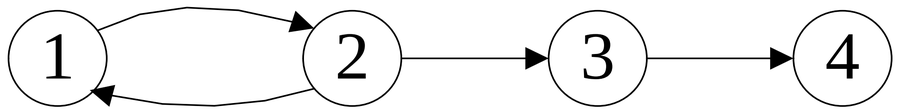
$$A^{+}(i, j) = \begin{cases} 1 & \text{如果从顶点i到顶点j的路径长度大于零} \\ 0 & \text{否则} \end{cases}$$

  - **G的自反传递闭包矩阵**

$$A^{*}(i, j) = \begin{cases} 1 & \text{如果从顶点i到顶点j的路径长度大于等于零} \\ 0 & \text{否则} \end{cases}$$

- **A^{+}和A^{*}的区别仅在对角线上的元素**
  - **当且**仅当包含顶点i**的回路**长度大于1时，$A^{+}(i, i) = 1$
  - **而从**顶点i**到**顶点i总有一条长度为0**的路径，因此**$A^{*}(i, i) = 1$总成立

计算传递闭包示例



$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

$$A^{+} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

$$A^{*} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

计算传递闭包算法的基本思路

- **修改Floyd算法就能求出邻接矩阵A的传递闭包矩阵A^+**
- **A的元素定义：** $$A(i,j)=\begin{cases}1 & i\neq j\text{且}<i,j>\in E(G)\\ 0 & \text{否则}\end{cases}$$
- **在Floyd算法中，除了删去使用数组Path的语句之外，还需把下面的语句：**

```
if (A[i][k] + A[k][j] < A[i][j])  A[i][j] = A[i][k] + A[k][j];
```

- **改为：**

```
if (A[i][j] == 0)
    if (A[i][k] + A[k][j] == 2)  A[i][j] = 1;
```

A[i][k]==1
A[k][j]==1

计算传递闭包算法的实现

```
void Graph::A_A+ (int n) {  // A --> A+
    for (int k = 0; k < n; k++)
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                if (A[i][j] == 0)
                    if (A[i][k] + A[k][j] == 2)  A[i][j] = 1;
}
void Graph::A+_A* (int n) {  // A+ --> A*
    for (int i = 0; i < n; i++)
        if (A[i][i] == 0)  A[i][i] = 1;
}
```

08 图

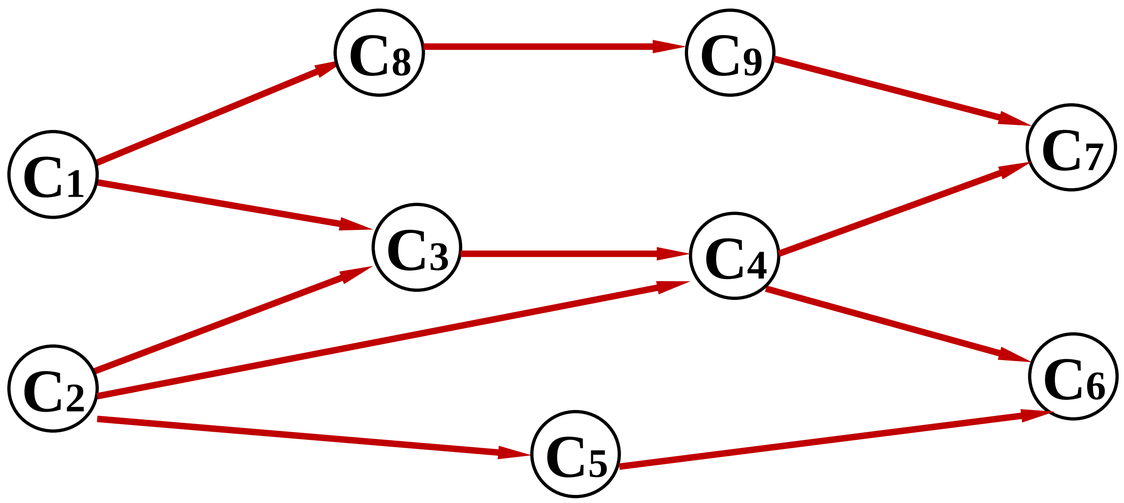
- 最短路径
- **拓扑排序**
- 关键路径

拓扑排序

- 计划、施工过程、生产流程、程序流程等都是“**工程**”。除很小的工程外，一般都把工程分为若干个叫做“**活动**”的子工程。完成这些活动，这个工程就可以完成
- 例如，计算机专业学生的学习就是一个工程，每一门课程的学习就是整个工程的一些活动。其中，有些课程要求先修课程，有些则不要求。这样，在有的课程之间有先后关系，有的课程可以并行地学习

课程之间的先后关系

课程代号	课程名称	先修课程
C_1	高等数学	
C_2	程序设计基础	
C_3	离散数学	C_1, C_2
C_4	数据结构	C_3, C_2
C_5	高级语言程序设计	C_2
C_6	编译方法	C_5, C_4
C_7	操作系统	C_4, C_9
C_8	普通物理	C_1
C_9	计算机原理	C_8

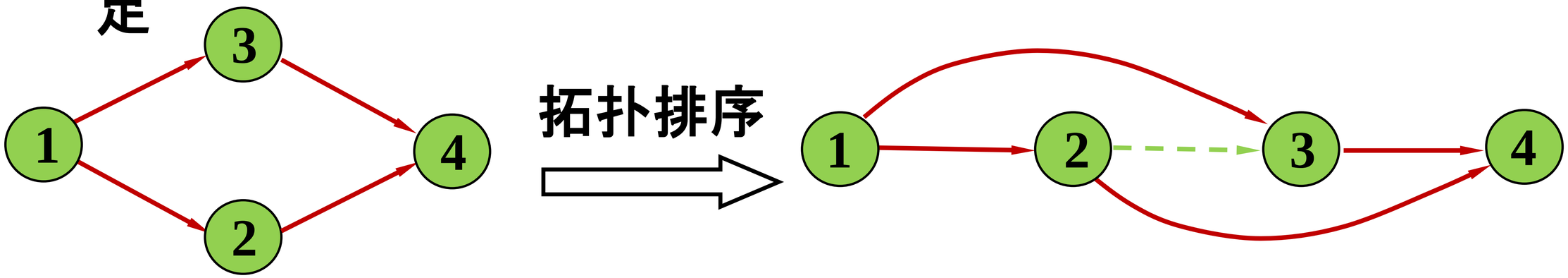


AOV网络

- 可以用有向图表示一个工程，用顶点表示活动，用有向边<V_i, V_j>表示活动V_i必须先于活动V_j进行，这种有向图叫做顶点表示活动(Activity On Vertices)的AOV网络
- 在AOV网络中不能出现有向回路，即有向环。如果出现了有向环，就意味着某项活动应以自己作为先决条件。因此，对给定的AOV网络，必须判断它是否存在有向环

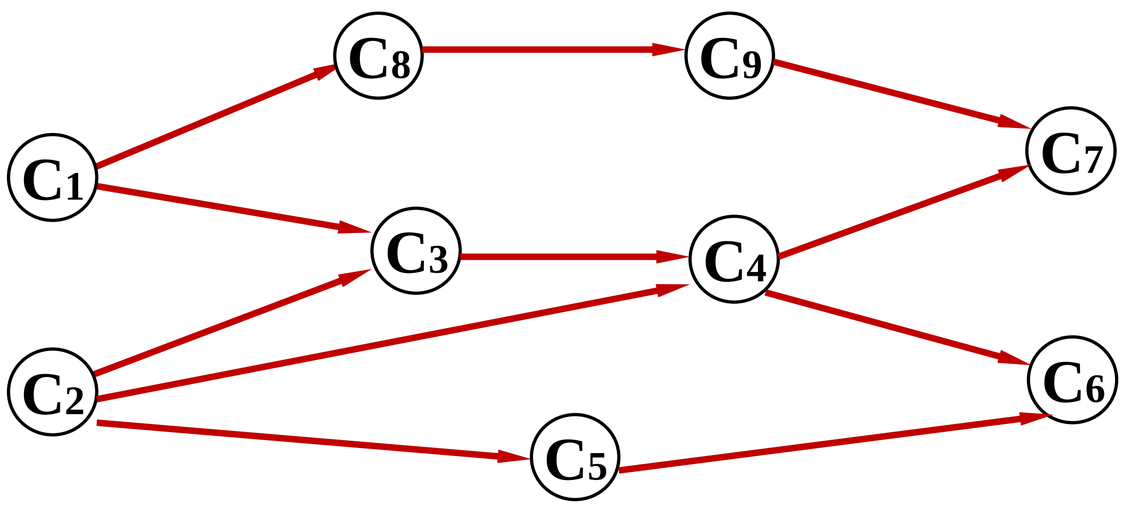
检测有向环的方法

- 检测有向环的一种方法是对AOV网络构造它的**拓扑排序**，即将各个顶点(即各个活动)排成一个线性有序序列，使得AOV网络中所有应存在的前驱和后继关系都能得到满足



- 如果通过拓扑排序能将AOV网络的所有顶点都排入一个拓扑有序的序列中，则该网络中必定不会出现有向环
- 如果AOV网络中存在有向环，此AOV网络所代表的工程是不可行的

对课程学习工程图进行拓扑排序



可能的拓扑有序序列为

C1, C2, C3, C4, C5, C6, C8, C9, C7

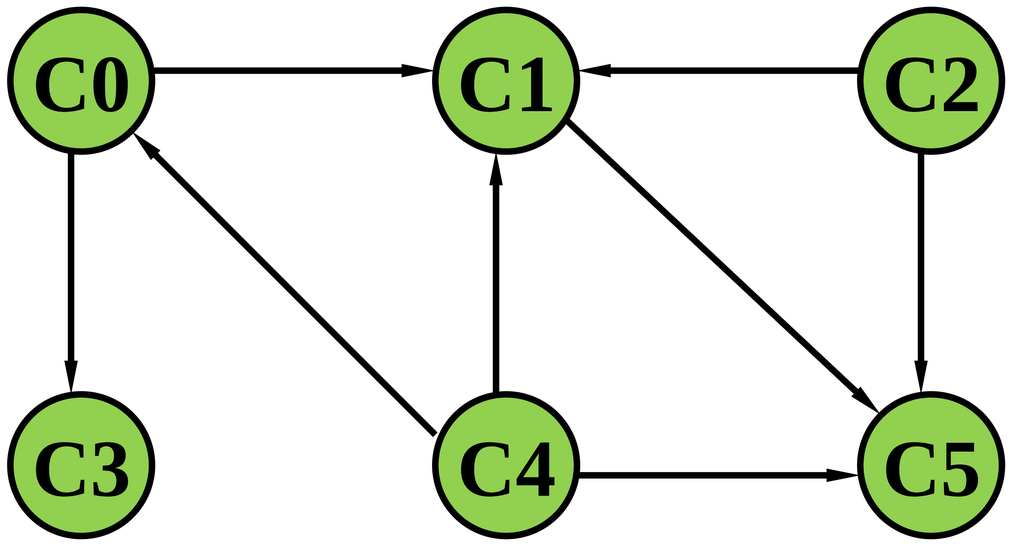
C1, C8, C9, C2, C5, C3, C4, C7, C6

……

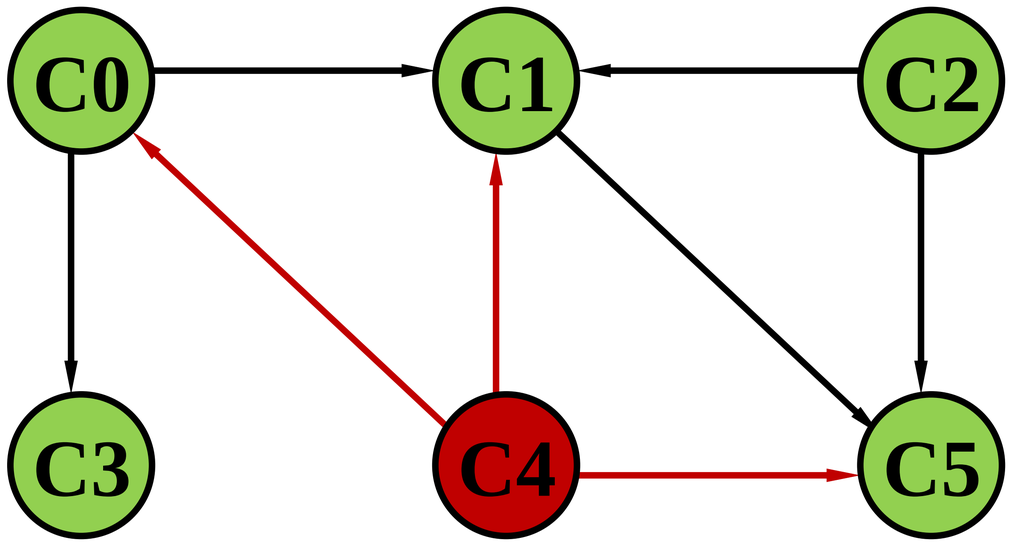
拓扑排序基本思路

- ① 输入AOV网络，令n为顶点个数
- ② 在AOV网络中选一个没有直接前驱的顶点，并输出
- ③ 从图中删去该顶点，同时删去所有它发出的有向边
- ④ 重复以上②和③步，直到
  - 全部顶点均已输出，拓扑有序序列形成，拓扑排序完成
  - 或图中还有未输出的顶点，但已跳出处理循环，则说明图中还剩下一些顶点，它们都有直接前驱，这时网络中必存在有向环

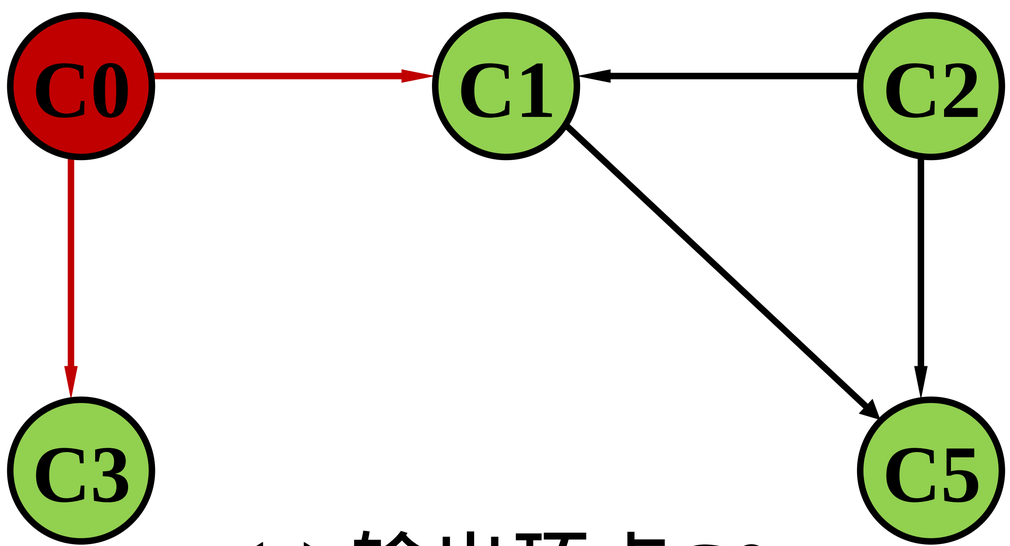
拓扑排序过程示例



(a) 有向无环图

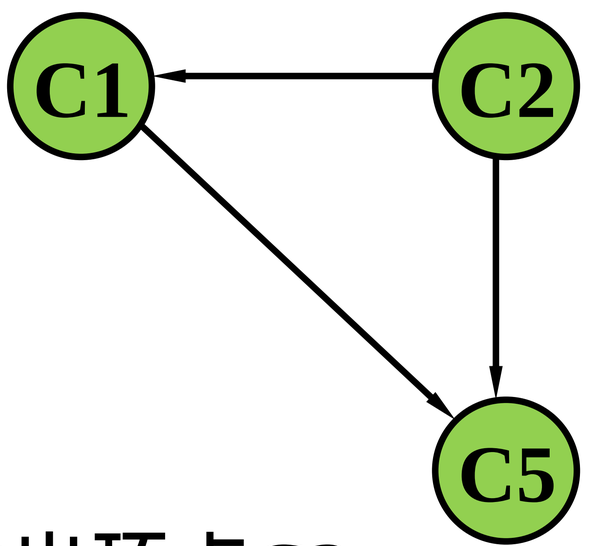


(b) 输出顶点C4



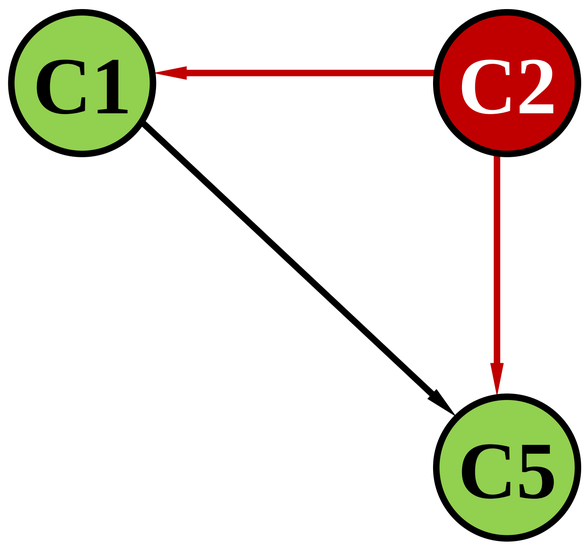
(c) 输出顶点C0



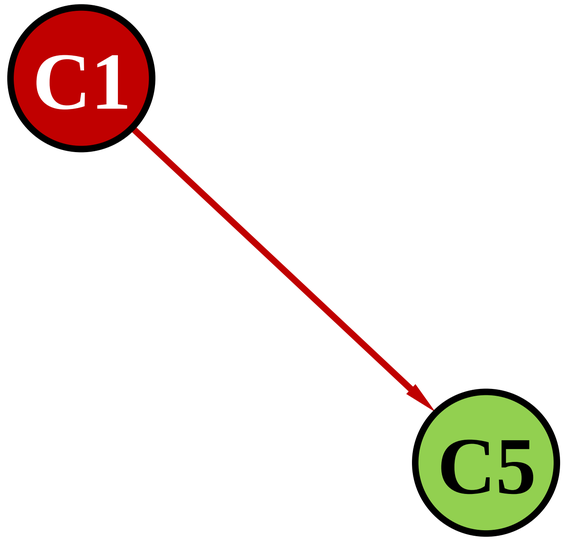


(d) 输出顶点C3

拓扑排序过程示例



(e) 输出顶点C2

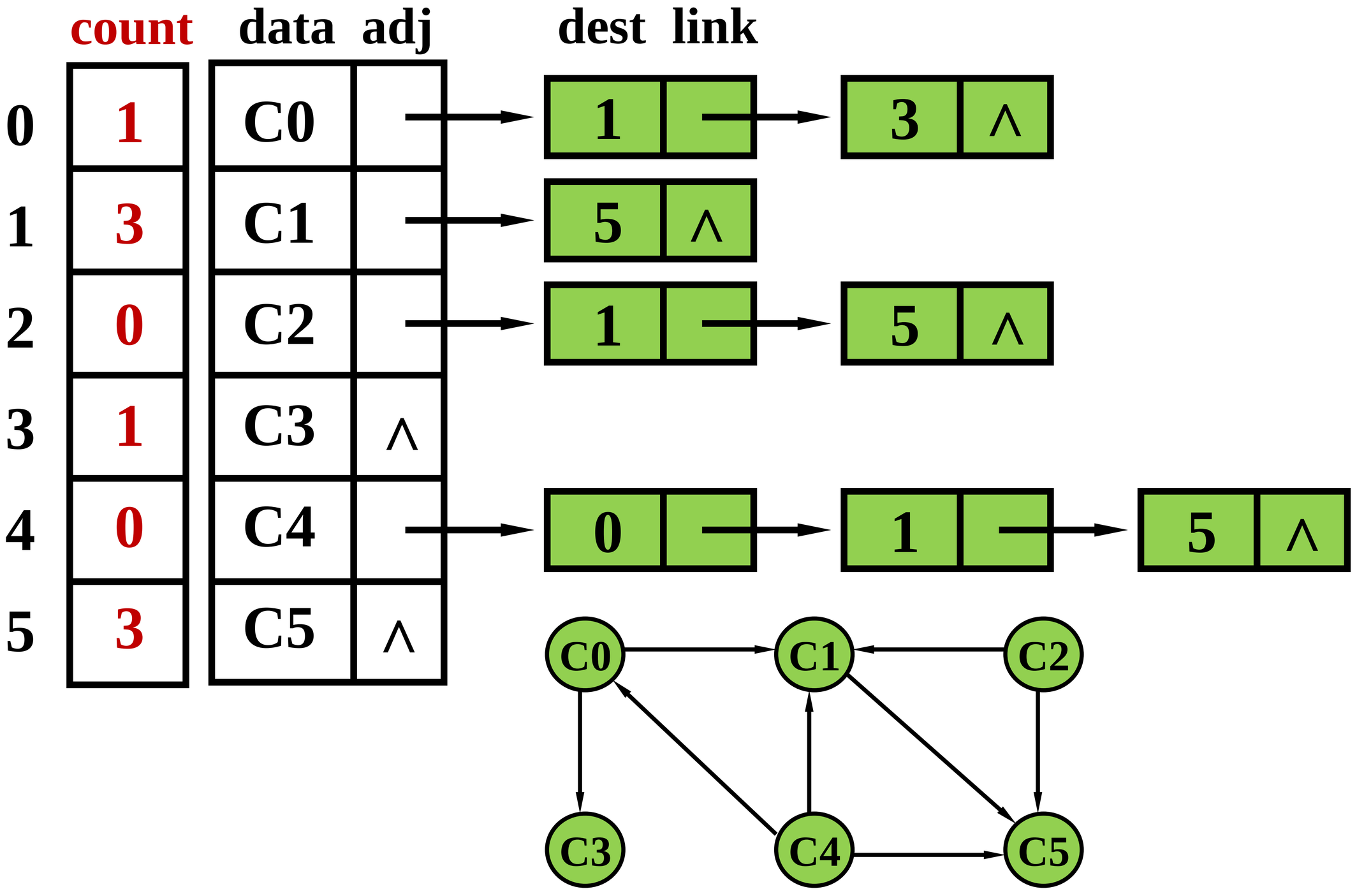


(f) 输出顶点C1

(g) 输出顶点C5

得到的拓扑有序序列为C4, C0, C3, C2, C1, C5，它满足图中给出的所有前驱和后继关系，对于本来没有这种关系的顶点，如C4和C2，也排出了先后次序关系

AOV网络及其邻接表表示



需要的数据结构

- 需要一个入度数组count[ ]，记录各顶点的入度，入度为零的顶点即无前驱顶点
- 在输入边数据时，每输入一条边<j, k>，就需要建立一个边结点，并将它链入相应边链表中，并更新入度数组
  - EdgeNode *p = new EdgeNode <int> (k); // 建立边结点，dest为k
  - p->link = NodeTable[j].adj;
  - NodeTable[j].adj = p; // 链入顶点j的边链表的前端
  - count[k]++; // 顶点k入度加一
- 需要一个存放入度为零的顶点的栈，供输出无前驱顶点

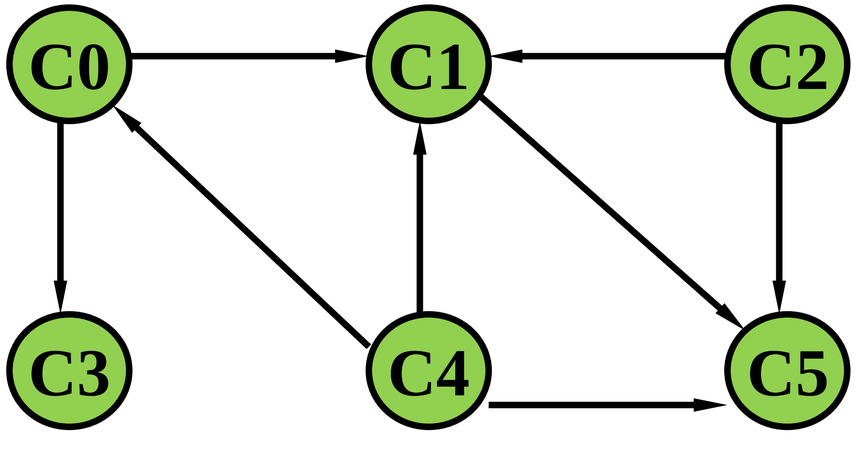
拓扑排序实现基本思路

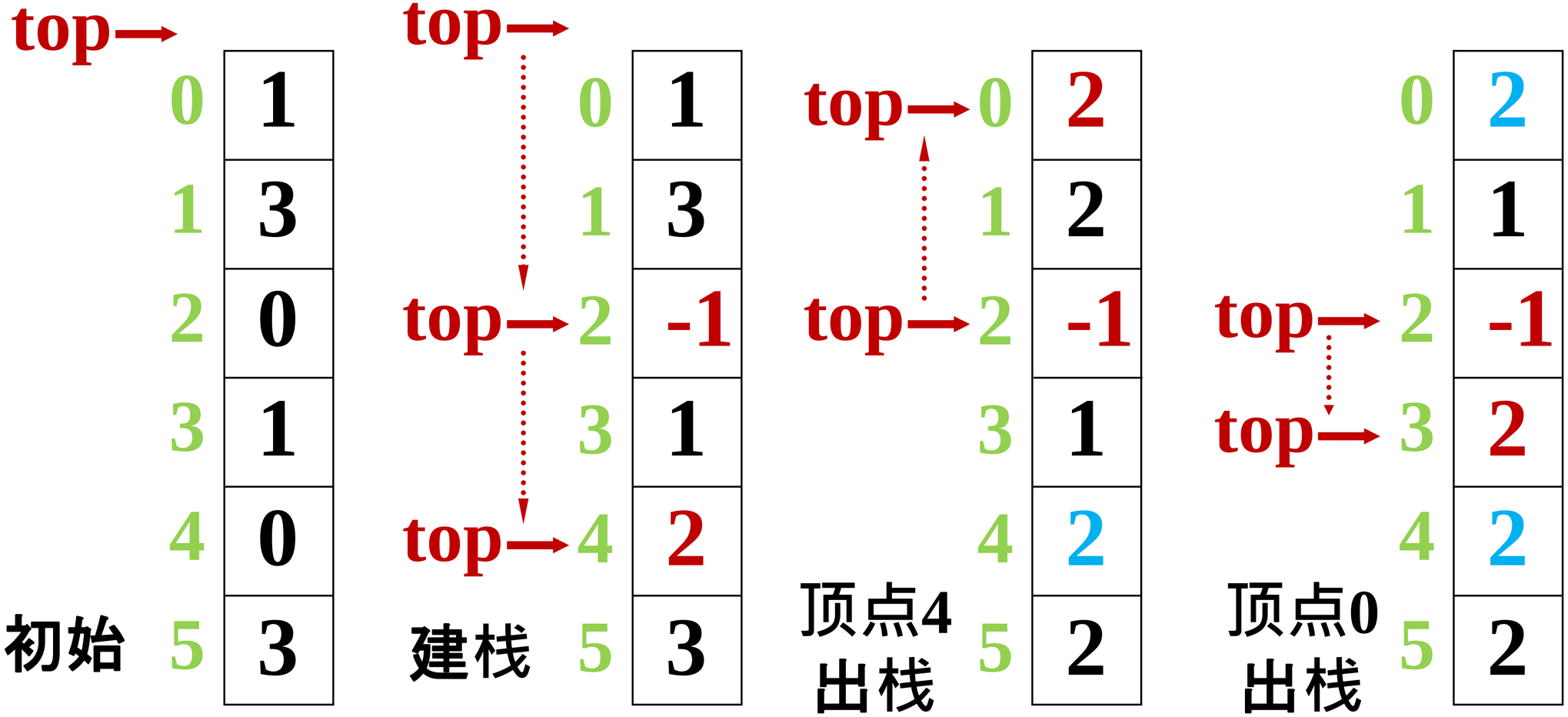
- 输入边数据，建立入度数组count和邻接表
- 建立入度为零的顶点栈
- 当入度为零的顶点栈不空时，重复执行：
  - 从顶点栈中退出一个顶点，并输出
  - 从AOV网络中删去这个顶点和它发出的边，边的终顶点入度减一
  - 如果边的终顶点入度减至0，则该顶点进入入度为零的顶点栈
- 如果输出顶点个数少于AOV网络的顶点个数，则网络中存在有向环，不存在拓扑排序结果

拓扑排序实现的改进

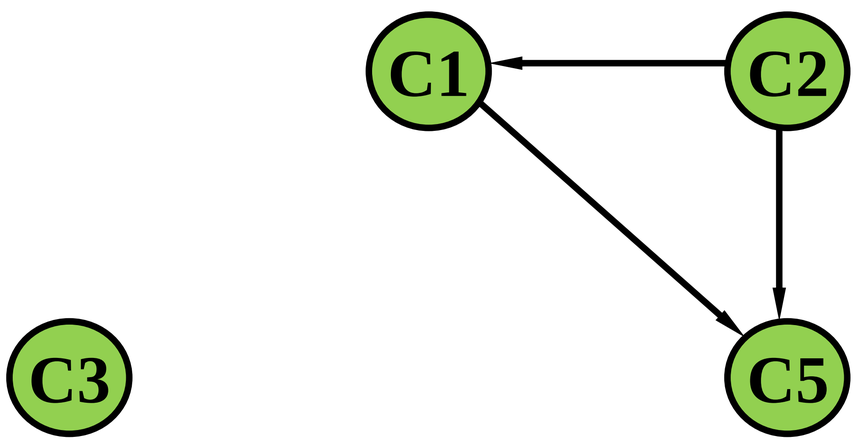
- 在实现时，为建立入度为零的顶点栈，可以不另外分配存储空间，直接利用顶点的入度count[]数组实现栈操作
  - 设立一个栈顶指针top指示当前栈顶元素在count[]数组中的位置，即某一个入度为零的顶点；栈初始化时，置top = -1
- 将顶点i进栈时执行以下操作：
  - count[i] = top; top = i; // top指向新栈顶i，原栈顶在count[i]中
- 退栈时执行以下操作：
  - j = top; top = count[top]; // 栈顶位置记为j，top退到次栈顶

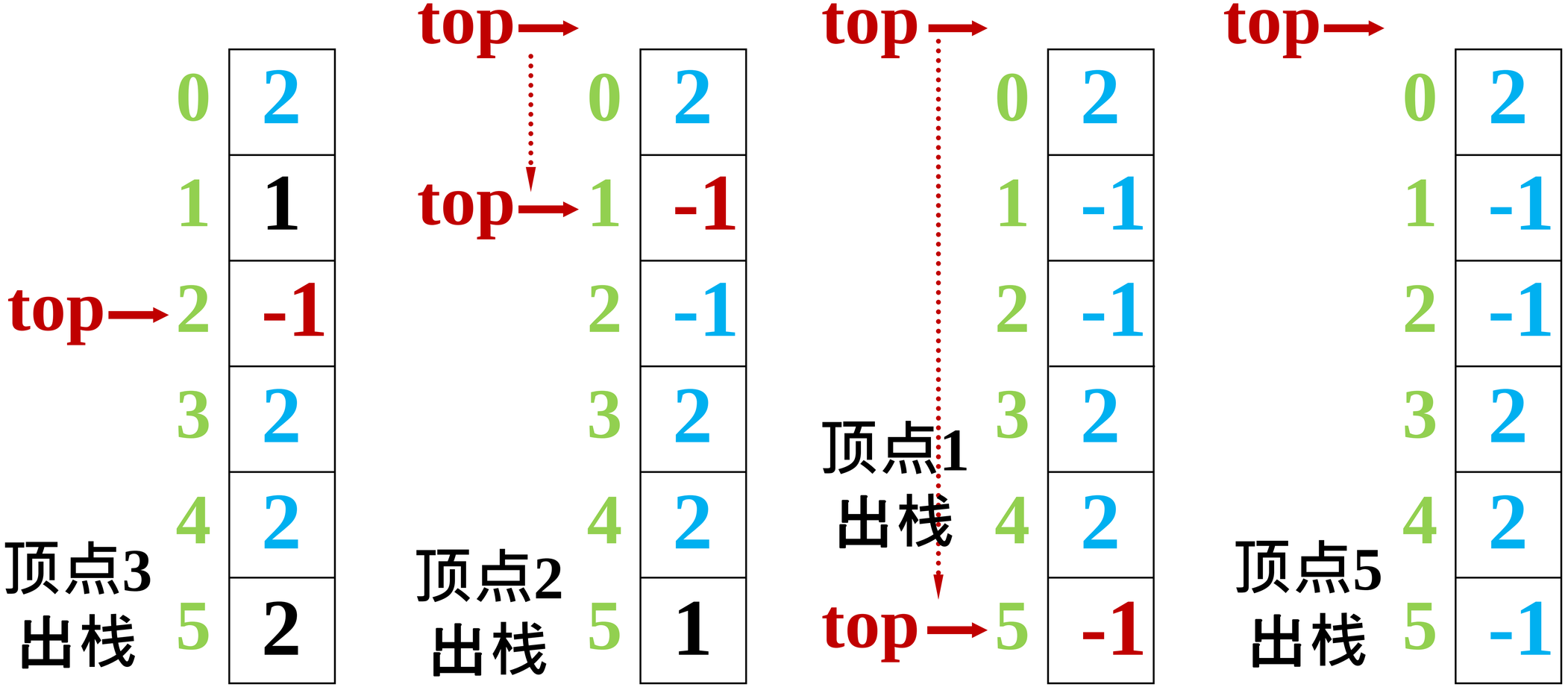
拓扑排序时入度为零的顶点栈在count[ ]中的变化示例





拓扑排序时入度为零的顶点栈在count[ ]中的变化示例





拓扑排序实现

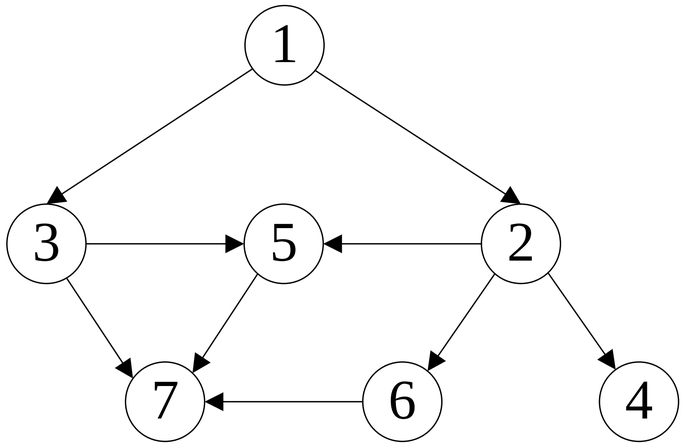
```
void TopologicalSort (Graph<string, float>& G ) {
    int i, j, k;  int top = −1; int n = G.VerticesNumber( ); int *count = new int[n];
    for (i = 0; i < n; i++) count[i] = 0;
    cin >> i >> j;  cout << endl;  // 入度数组和邻接表初始化
    while (i > −1 && i < n && j > −1 && j < n) {
        G.InsertEdge(i, j, 0);  count[j]++;   cin >> i >> j;  cout << endl;
    }
    for (i = 0; i < n; i++)  // 入度为零顶点进栈
        if (count[i] == 0) { count[i] = top;  top = i; }
    for (i = 0; i < n; i++)  // 期望输出n个顶点
        if (top == −1) { cout << “网络中有回路” << endl; return; }  // 中途栈空
        else {
            j = top;  top = count[top];  cout << j << endl;  // 出栈
            k = G.GetFirstNeighbor (j);
            while (k != −1) {  // 扫描出边表
                if (−−count[k] == 0) { count[k] = top;  top = k; }  // 入度减至零，进
栈
                k = G.GetNextNeighbor(j, k);
            }
        }
}
```

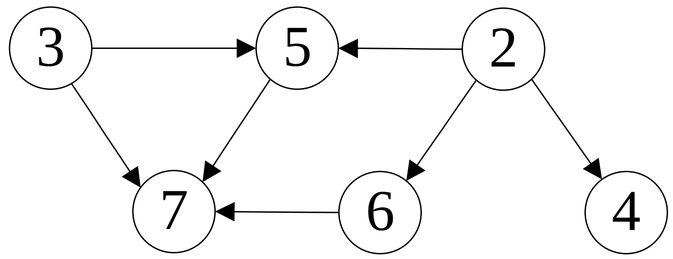
拓扑排序性能分析

- 若AOV网络有|V|个顶点和|E|条边，在拓扑排序的过程中
  - 建立邻接表的时间为O(|E|)
  - 搜索入度为零的顶点，建立链式栈所需要的时间是O(|V|)
  - 每个顶点进一次栈，出一次栈，共输出|V|次
  - 顶点入度减一的运算共执行|E|次
  - 所以总的时间复杂度为O(|V|+|E|)

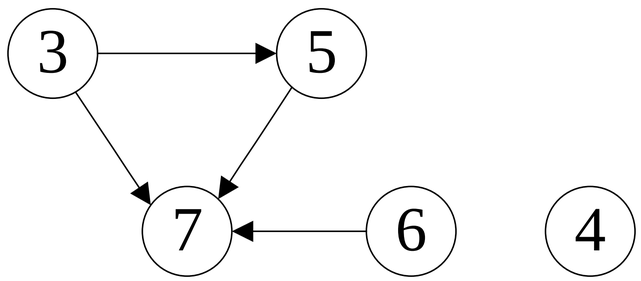
练习4

- **求下图的一个拓扑排序序列**

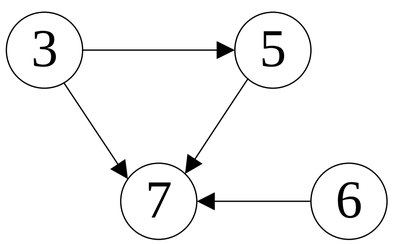




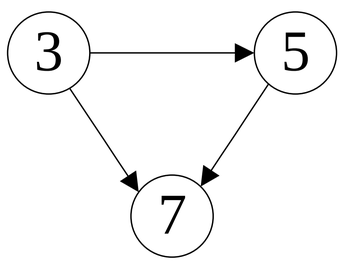
输出1



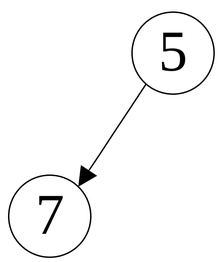
输出2



输出4



输出6



输出3



输出5

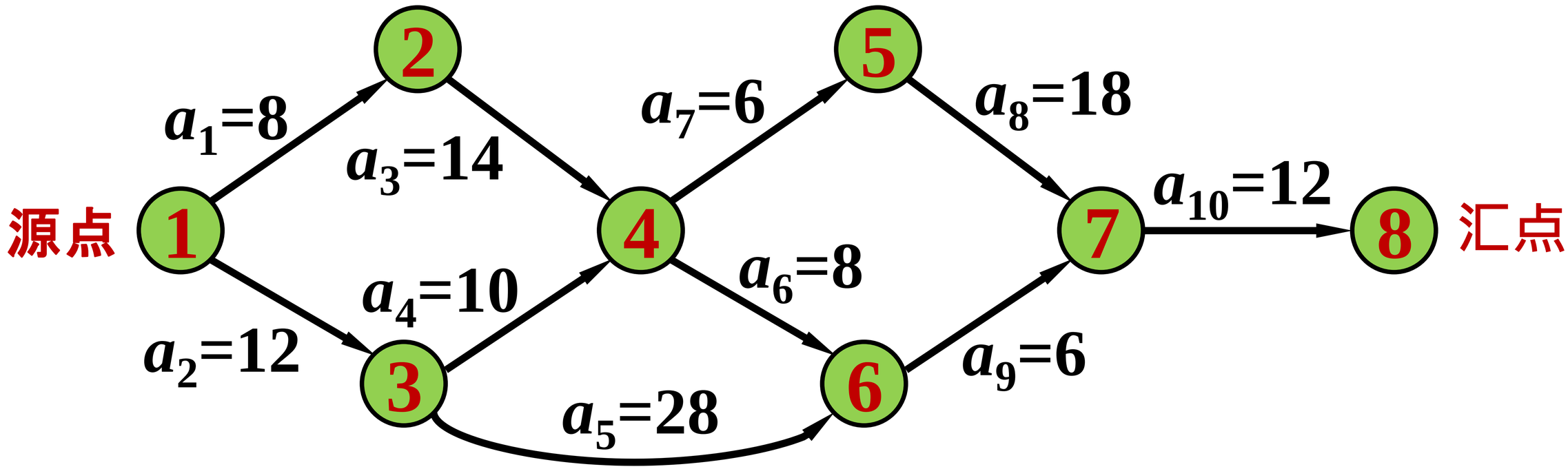
输出7

08 图

- 最短路径
- 拓扑排序
- 关键路径

AOE网络

- 在无环的带权有向图G中，用有向边表示一个工程中的**活动**，用边上权值表示**活动持续时间**，用顶点表示**事件**，则G叫做用边表示活动(Activity On Edges)的**AOE网络**

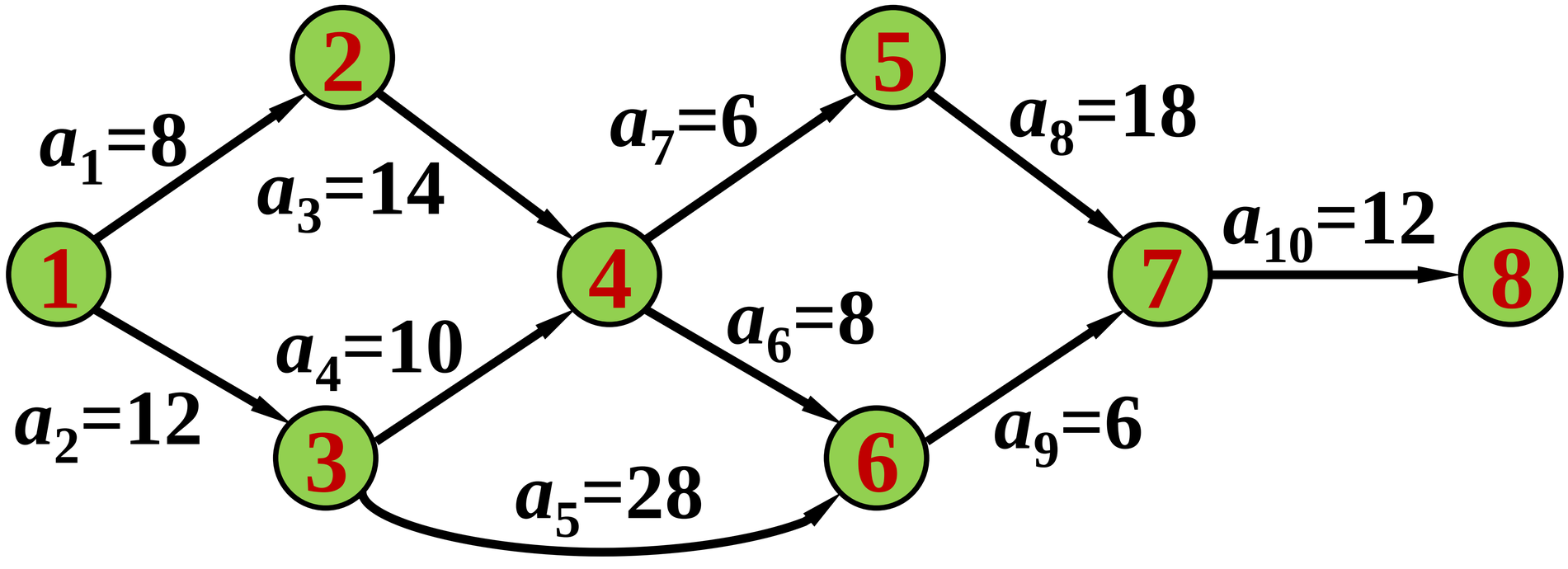


关键路径

- AOE网络在某些工程估算方面非常有用
  - 完成整个工程至少需要多少时间(假设网络中没有环)？
  - 为缩短完成工程所需的时间，应当加快哪些活动？
- 从源点到汇点的有向路径可能不止一条，它们的长度也可能不同。完成不同路径的活动所需的时间虽然不同，但只有各条路径上所有活动都完成，整个工程才算完成
- 完成整个工程所需的时间取决于从源点到汇点的最长路径长度，即在这条路径上所有活动的持续时间之和，这条路径长度最长的路径就叫做关键路径(Critical Path)

关键活动

- 要找出关键路径，必须找出关键活动，即不按期完成就会影响整个工程完成的活动
- 关键路径上的所有活动都是关键活动，因此只要找到关键活动，就可以找到关键路径
- 可能有多条关键路径，故不是任一关键活动加速一定能使工程提前，想使工程提前，要所有关键路径都能缩短



找出关键活动

- 事件V_i的最早可能开始时间**ee[i]**
  - 是从源点V_1到顶点V_i的最长路径长度
- 事件V_i的最迟允许开始时间**le[i]**
  - 是在保证汇点V_n在ee[n]时刻完成的前提下，事件V_i的允许的最迟开始时间，le[i]等于ee[n]减去V_i到V_n的最长路径长度
- 活动a_x的最早可能开始时间**ea[x]**
  - 设活动a_x在边$<V_j, V_k>$上，则ea[x]是从源点V_1到顶点V_j的最长路径长度，因此，ea[x] = ee[j]
- 活动a_x的最迟允许开始时间**la[x]**
  - la[x]是在不会引起时间延误的前提下，该活动允许的最迟开始时间，设活动a_x在边$<V_j, V_k>$上，la[x] = le[k] - dur(< j, k>)

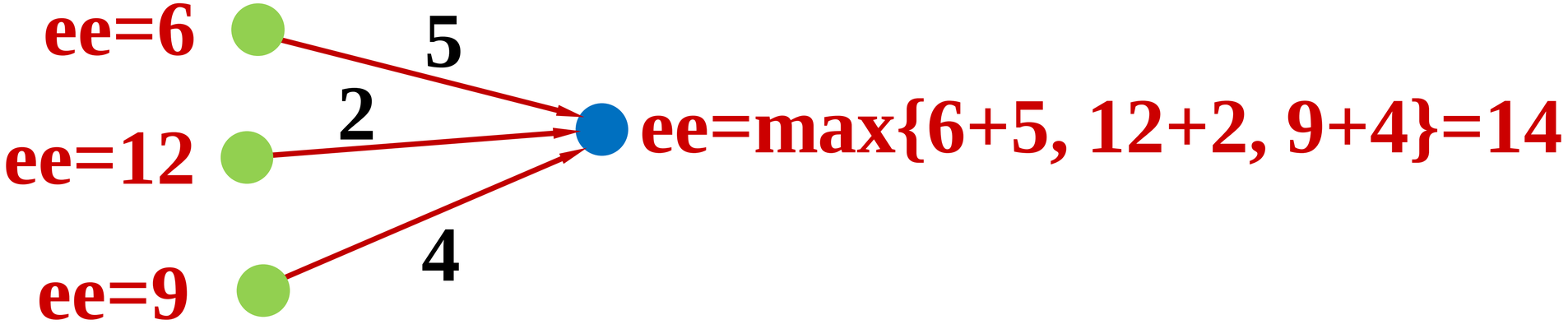
求关键活动

- 时间余量(又叫松弛时间)la[x]-ea[x]
  - 表示活动a_x的最早可能开始时间和最迟允许开始时间的时间余量
  - 在不增加完成工程所需总时间的情况下，活动a_x可以拖延的时间
  - 若la[x] == ea[x]，则表示活动a_x是没有时间余量的关键活动
  - 若a_x拖延的时间大于la[x]-ea[x]，则关键路径就可能发生变化

- 为找出关键活动，需要求各个活动的ea[x]与la[x]，以判别是否la[x] == ea[x]
- 为求得ea[x]与la[x]，需要求得从源点V_1到各个顶点V_i的ee[i]和le[i]

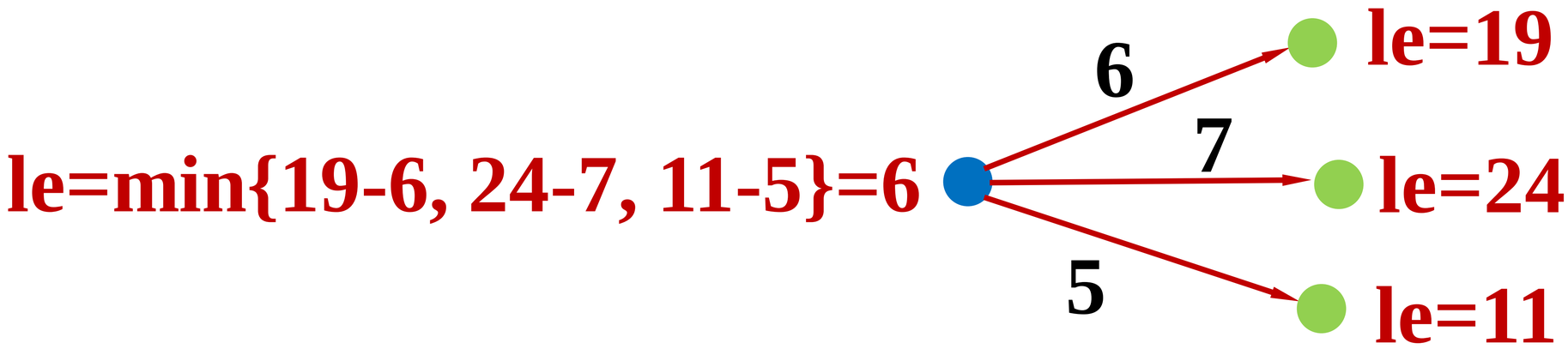
求ee[i]的递推公式

- 从ee[1] = 0开始，按拓扑排序顺序向前递推
  - ee[i] = max{ee[j] + dur(<V_j, V_i>)}
  - <V_j, V_i> ∈ S1, S1是所有指向V_i的有向边<V_j, V_i>的集合



求le[i]的递推公式

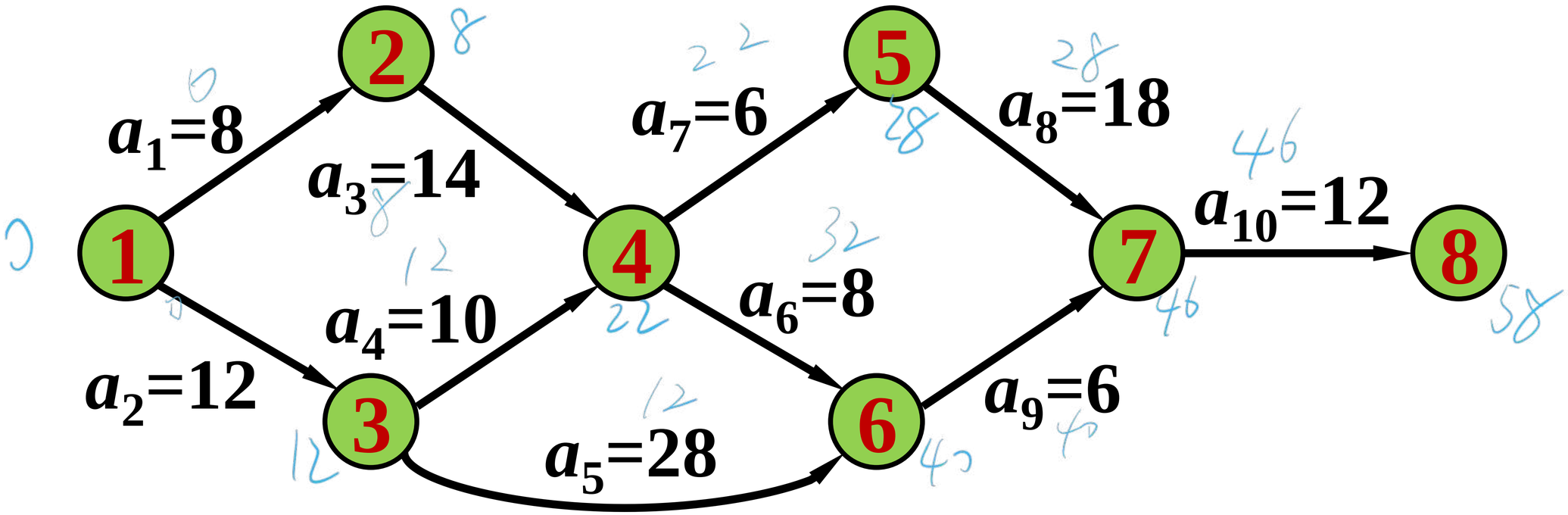
- 从le[n] = ee[n]开始，按拓扑排序的反向递推
  - le[i] = min{le[j] - dur(<V_i, V_j>)}
  - <V_i, V_j> ∈ S2, S2是所有源自V_i的有向边<V_i, V_j>的集合



计算过程示例

	1	2	3	4	5	6	7	8
ee	0	8	12	22	28	40	46	58
le	0	8	12	22	28	40	46	58

	1	2	3	4	5	6	7	8	9	10
ea	0	0	8	12	12	22	22	28	40	46
la	0	0	8	12	12	32	22	28	40	46



求关键活动实现

- 为简化算法，假定：
  - 在求关键路径之前已经对各顶点实现了拓扑排序，并按拓扑有序的顺序对各顶点重新进行了编号
  - 在邻接表中边链表的结点内增加一个cost域， 记录该边上的权值

求关键活动实现

```
void CriticalPath(Graph <string, float> &G) {
    int i, j, k;  float e, l, w;
    int n=G.VerticesNumber( );
    float *ee = new float [n];
    float *le = new float [n];
for (i = 0; i < n; i++)  ee[i] = 0;
for (i = 0; i < n; i++) {  // 顺向计算ee[ ]
    j = G.GetFirstNeighbor(i);
    while (j != -1) {
        w = G.GetWeight(i, j);
        if (ee[i] + w > ee[j])  ee[j] = ee[i] + w;
        j = G.GetNextNeighbor(i, j);
    }
}
le[n-1] = ee[n-1];
```

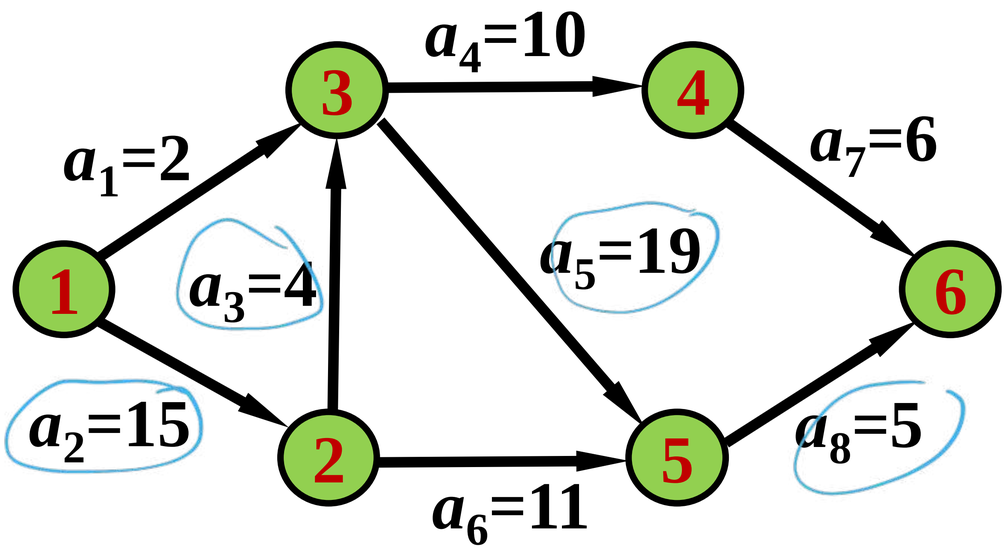
求关键活动实现

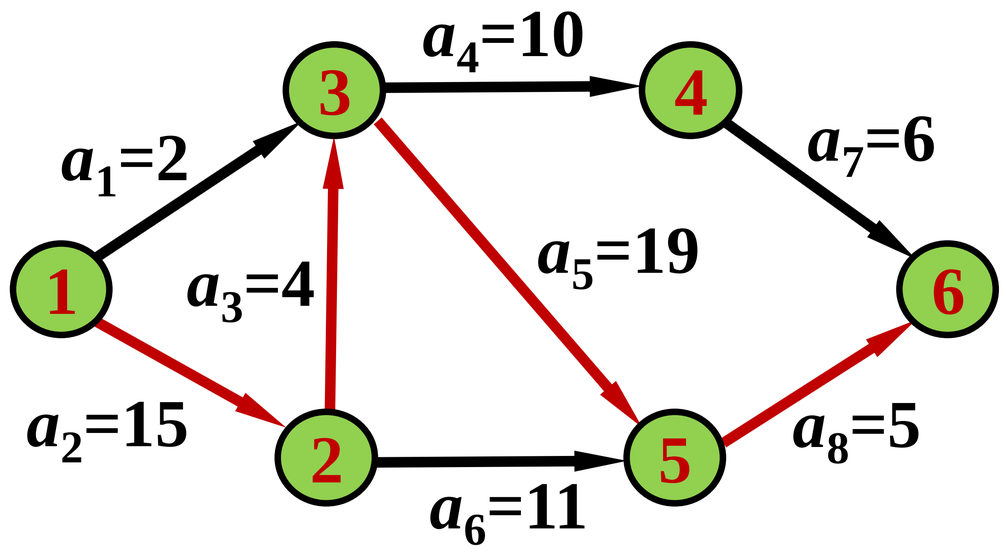
```
for (j = n-2; j > 0; j--) {  // 逆向计算le[ ]
    k = G.GetFirstNrighbor(j);
    while (k != -1) {
        w = G.GetWeight(j, k);
        if (le[k] - w < le[j])  le[j] = le[k] - w;
        k = G.GetNextNeighbor(j, k);
        }
    }
    for (i = 0; i < n; i++) {  // 求各活动的e和l
        j = G.GetFirstNeighbor(i);
        while (j != -1) {
            e = ee[i];  l = le[j] - G.GetWeight(i, j);
            if (l == e)  cout<<"<"<<i<<","<< j<<">"<<endl;
            j = G.GetNextNeighbor(i, j);
        }
    }
}
```

ea la

练习5

- 确定哪些是关键活动？哪些活动加速可使工程提前？





	1	2	3	4	5	6
ee	0	15	19	29	38	43
le	0	15	19	37	38	43

	1	2	3	4	5	6	7	8
ea	0	0	15	19	19	15	29	38
la	17	0	15	27	19	27	37	38

本章小结

- 最短路径
- 拓扑排序
- 关键路径

作业

- 为判断图中是否存在环(回路)给出尽量多的方法，说明每一个方法的具体思路
- 若表示一个有向图的邻接矩阵中对角线以下的元素均为0，则该图的拓扑有序序列一定存在吗？为什么？